

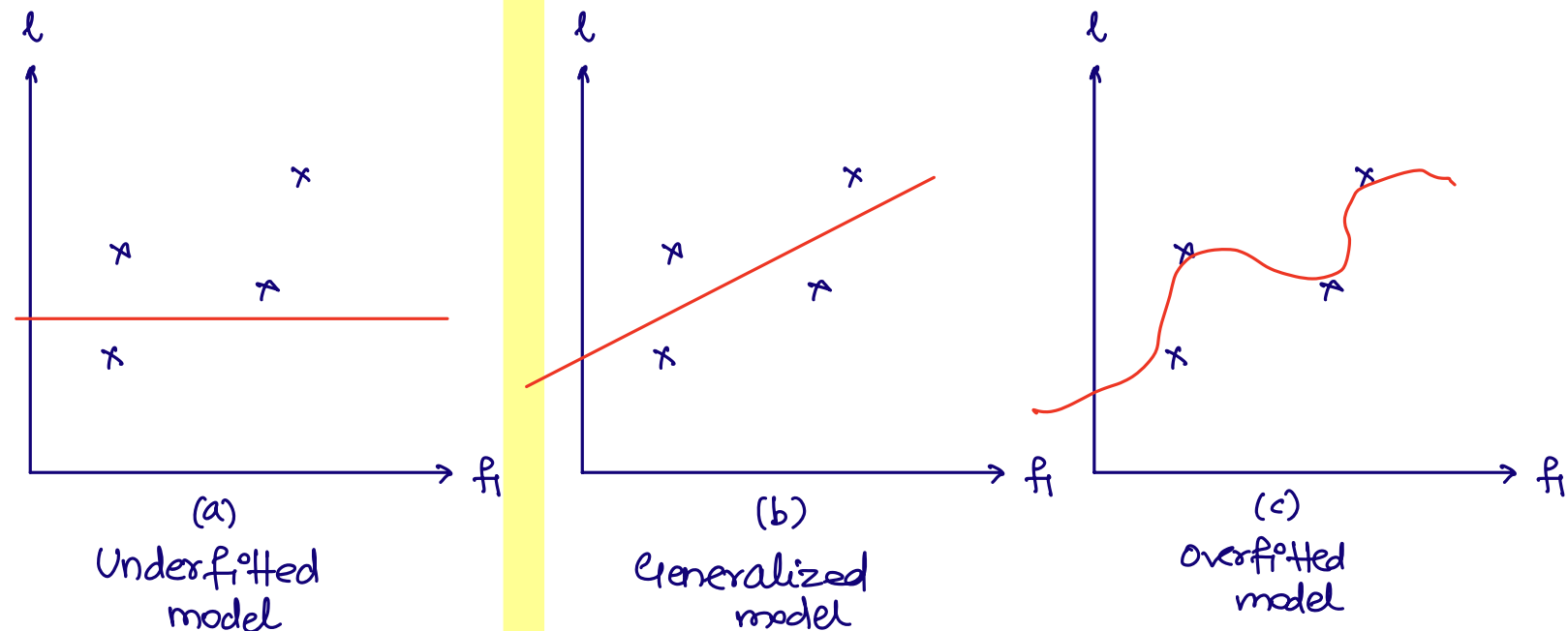
Significance Level (SL): It defines how much % error is tolerable in trained model, such that the model is in compliance with SLA.

— 0.05 —
 α values

Default values: 0.05, 0.01, 0.1

SAMPLES
↓
STATISTICS
↓
APPROXIMATION

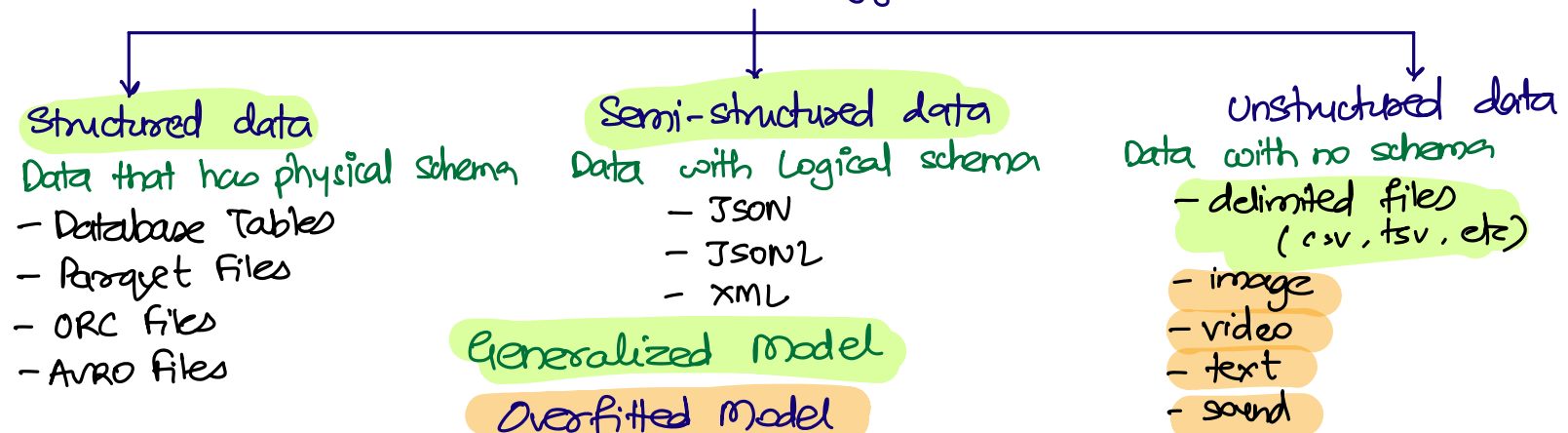
95% depends on DOMAIN to decide which model to DEPLOY



5% depends on TYPE OF SAMPLE you deal with.

Regression
use case

Types of data in
Data Engg



Scope table

Scope	ML	DL
Structured Data - Database Tables - Parquet files - AVRO - ORC	- faster → training → deployment → prediction	- slower → training - faster → deployment → prediction
Semi-structured Data - JSON JSONL - XML	- Easy to implement - High Accuracy*	- chances of OVERFITTING is more
Unstructured Data - Delimited files - csv - tsv - pipe sep	- Project COST is less as compared to DL	ANN Artificial Neural Network
- Images - Video - Sound - Waveforms - Text	☹️ I cant HELP! X X X X Limited support using NLP technique + statistics	😊 I can help! CNN 2D VisionTransformer CNN 3D VisionTransformer } CNN 1D, RNN, LSTM } CNN(1D), RNN, LSTM or Transformers

Neural Networks

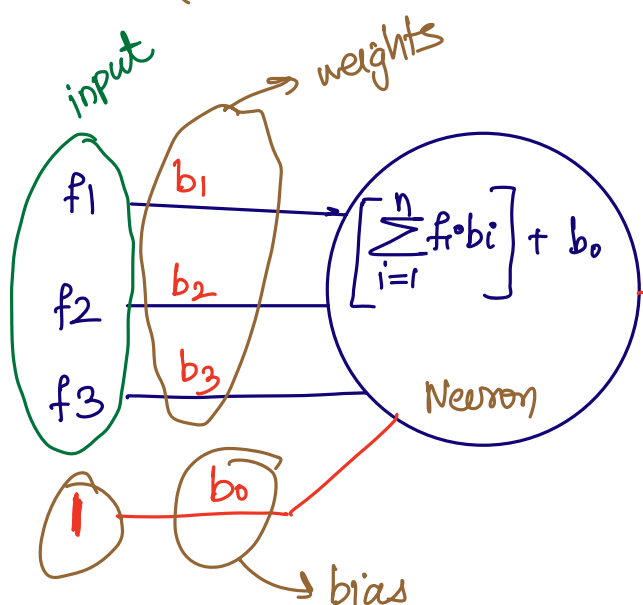
- Just another way of **CREATING AND REPRESENTING** an intelligent model.
- inspired from human brain (neuron)

Perceptron [ARTIFICIAL NEURON] → Applicable for Supervised Learning use-cases → Regression
→ Classification

- fundamental unit of deep NN.

- credits

Frank Rosenblatt (1958)
(Classical Perceptron) (regression)



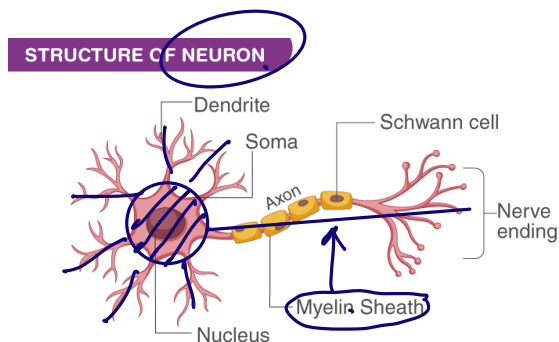
continuous ND
0 slope

slope intercept formula

$$y = mx + C$$

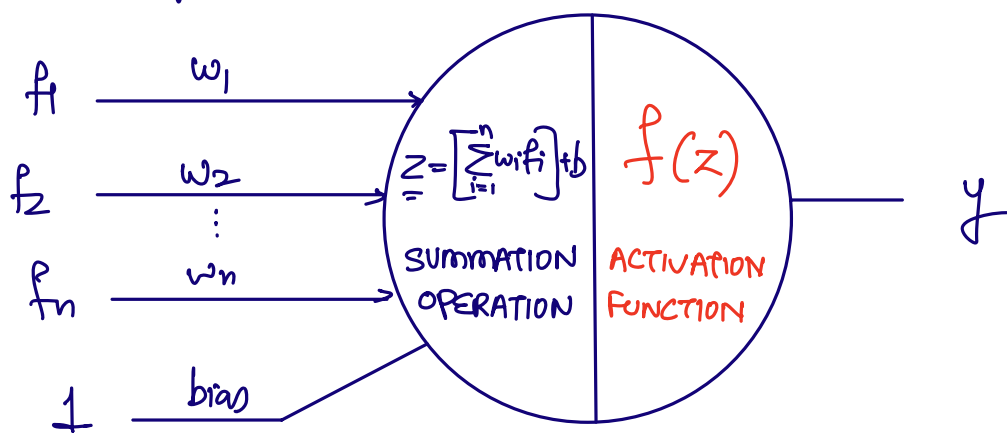
$y = b_0 + b_1 x_1 + b_2 x_2 + \dots + b_n x_n$

Annotations: b_0 is labeled 'intercept', $b_1, b_2, \dots, b_n$ are labeled 'weights', and $x_1, x_2, \dots, x_n$ are labeled 'features'.



- credits (1969)

Minsky & Papert (modified Classical Perceptron)



Activation function helps neurons to introduce **NON-LINEARITY** (giving categorical or discrete outputs)

① Pass-through function | linear activation function

$$f(z) = z \dots \text{used for regression use-cases}$$

② Sigmoid f^n (emits probability values) (0 - 1)

$$f(z) = \frac{1}{1 + e^{-z}} \rightarrow \text{used to solve BINARY CLASSIFN}$$

threshold $\begin{cases} 0 \text{ to } 0.5 \rightarrow 0 \\ > 0.5 \rightarrow 1 \end{cases}$

Q

dataset containing 1 record

f_1	f_2	f_3
-2.1	1.2	0.6

given weights

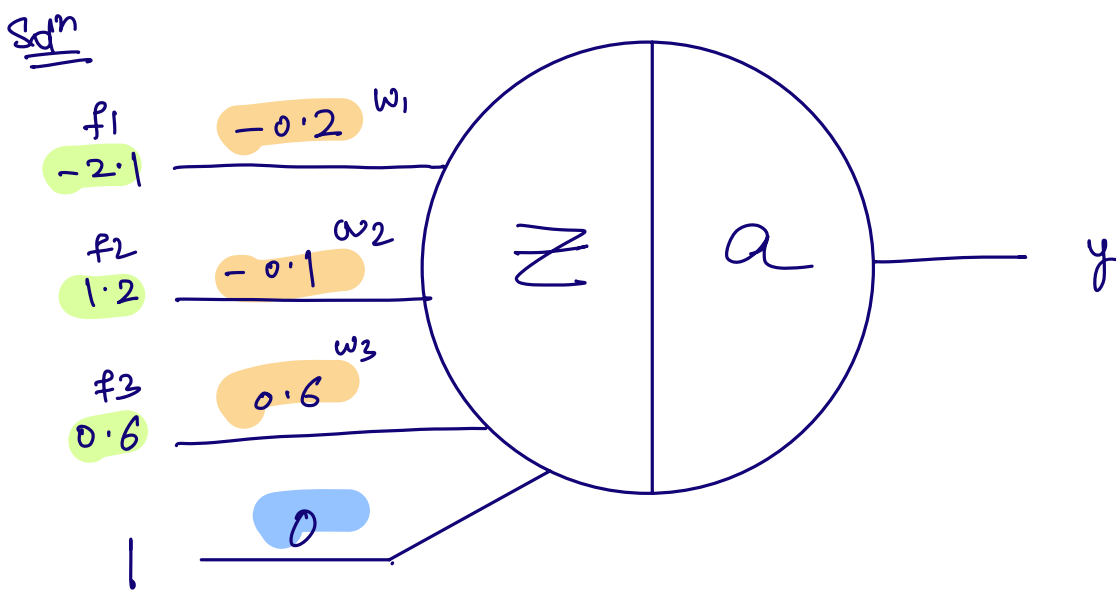
w_1	w_2	w_3
-0.2	-0.1	0.6

given bias = 0

using perceptron theory, find the o/p y if,

① y is continuous

② y is binary categorical.



① if y is continuous

② calc $z = \left[\sum_{i=1}^n w_i f_i \right] + b$

$$= [w_1 f_1 + w_2 f_2 + w_3 f_3] + \text{bias}$$

$$= [(-2.1)(-0.2) + (1.2)(-0.1) + (0.6)(0.6)] + 0$$

$$= 0.66$$

③ Calc $a \rightarrow$ using pass through function

$$f(z) = z$$

$$\boxed{a = 0.66}$$

② if y is binary categorical

$$\textcircled{a} \text{ calc } z = \left[\sum_{i=1}^n w_i f_i \right] + b$$

$$= [w_1 f_1 + w_2 f_2 + w_3 f_3] + \text{bias}$$

$$= [(-2.1)(-0.2) + (1.2)(-0.1) + (0.6)(0.6)] + 0$$

$$= 0.66$$

③ Since y is expected binary categorical

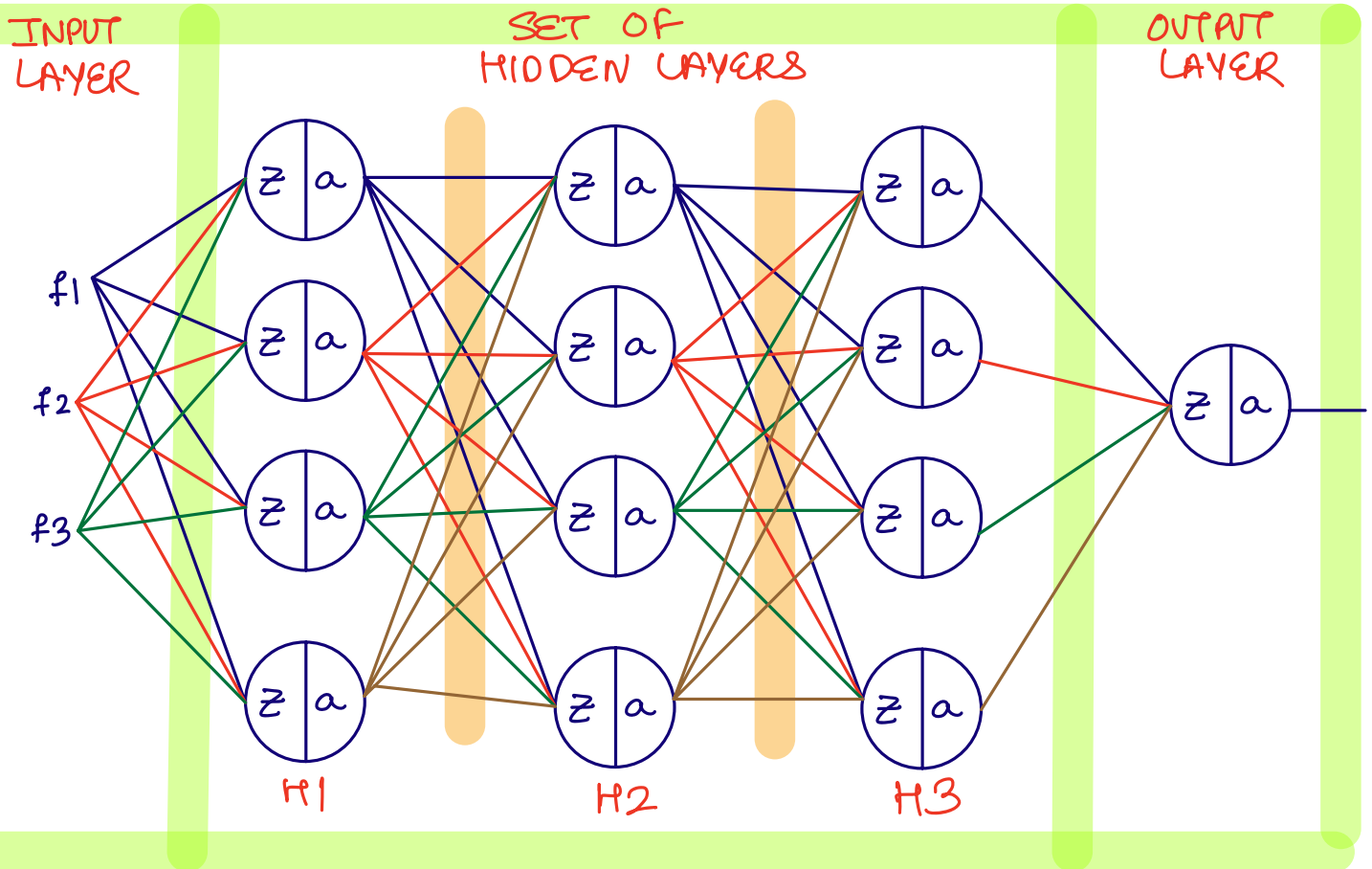
$$f(z) = \frac{1}{1 + e^{-z}}$$

$$= \frac{1}{1 + e^{-(0.66)}}$$

$$= 0.6592$$

using threshold rule $\longrightarrow 0.6592 > 0.5$
 $\therefore \textcircled{1}$

MLP (Multi-layer Perceptron) - or - ANN (Artificial Neural Network)



① a typical ANN consists of 3 layers

① Input layer

② Set of Hidden layers

③ output layer

② Number of units for i/p layer = No of features _{cols}

③ Number of units for o/p layer = YET TO DISCUSS :-

④ Hidden layers are Elastic

5x13

Any ANN training composed of 3 steps

✓ ① FORWARD PROPOGATION

✓ ② ERROR CALCULATION

✓ ③ GRADIENT CALCULATION

BACK PROPOGATION

Pre-req

Error function
↓
formula to calc
error of entire set
↓
training set
testing set
entire dataset

and
Loss function
↓
formula to calc
error of a
single record

ground truth ↓ y	model prediction ↓ $\hat{y}$	$(\hat{y} - y)$ loss
2	3	1
1	1	0
7	8	1
7	7	0

$$\text{Error} = \frac{\sum \text{loss}}{n_{\text{records}}} = \frac{1 + 0 + 1 + 0}{4} = 0.5$$

For regression

① Mean Squared Error = $\frac{\sum_{i=1}^n (y - \bar{y})^2}{n}$
or
MSE

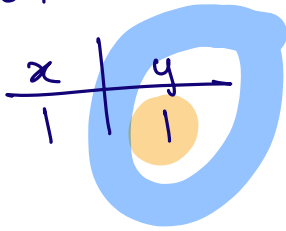
② Mean Absolute Error = $\frac{\sum_{i=1}^n |y - \bar{y}|}{n}$
or
MAE

Forward Propagation

Forward propagation is all about performing two operations:

- ① Calc z (summation operation) (summation function)
- ② Calc a (activation operation) (activation function)

Q dataset containing one record



x	y
1	1

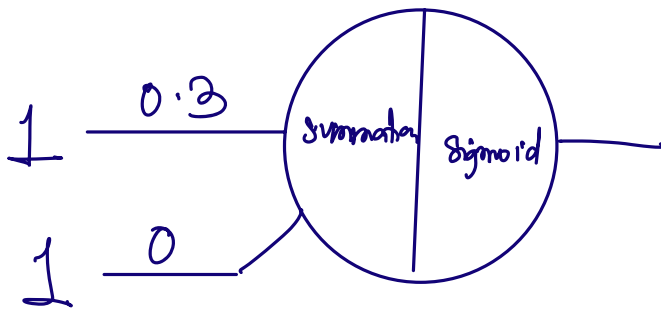
$$w_x = 0.3$$

$$b = 0$$

$$a = \text{sigmoid}$$

error $\rightarrow$ mse

perform forward propagation and error calc.



Solⁿ

① Forward propagation

$$\begin{aligned}\text{a) calc } z &= \left[\sum_{i=1}^n w_i f_i \right] + b \\ &= (0.3)(1) + 0 \\ &= \underline{\underline{0.3}}\end{aligned}$$

⑥ calc $a \rightarrow$ sigmoid

$$a = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-0.3}} = 0.5744 \rightarrow \bar{y}$$

⑦ Error calc

$$E = \frac{\sum_{i=1}^N (y - \bar{y})^2}{n} = \frac{(1 - 0.5744)^2}{1} = 0.1811 //$$

In terms of Statistical modelling

The goal of training an ANN is to IDENTIFY / CONVERGE
weights & bias such that the model achieves the required
evaluation metric threshold.

Sir Geoffrey Hinton → Godfather of AI

Algorithm: **Backpropagation**. Neural network learning for classification or prediction, using the backpropagation algorithm.

Input:

- D , a data set consisting of the features and their associated target values; label
- η , the learning rate; (hyperparameter) (0-1) (4 point precision) 0.0001
- $network$, a multilayer feed-forward network.

Output: A trained neural network.

Method:

- Initialize all weights and biases in $network$;
- while terminating condition is not satisfied {
- for each training tuple X in D {

(4) // Propagate the inputs forward:

(5) for each input layer unit j {

(6) $\{O_j = I_j\}$ // output of an input unit is its actual input value

(7) for each hidden or output layer unit j {

(8) $I_j = \sum_i w_{ij} O_i + \theta_j$ // compute the net input of unit j with respect to the previous layer, i

(9) $a_j = \frac{1}{1+e^{-I_j}}$; // compute the output of each unit j

(10) // Backpropagate the errors:

(11) for each unit j in the {output layer}

(12) $\rightarrow Err_j = O_j(1-O_j)(T_j-O_j)$; // compute the error

(13) for each unit j in the hidden layers, from the last to the first hidden layer

(14) $\rightarrow Err_j = O_j(1-O_j) \sum_k Err_k w_{jk}$; // compute the error with respect to the next higher layer, k

(15) for each weight w_{ij} in $network$ {

(16) $\rightarrow \Delta w_{ij} = (\eta Err_j O_i)$; // weight increment

(17) $\rightarrow w_{ij} = w_{ij} + \Delta w_{ij}$; // weight update

(18) for each bias θ_j in $network$ {

(19) $\Delta \theta_j = (\eta Err_j)$; // bias increment

(20) $\theta_j = \theta_j + \Delta \theta_j$; // bias update

(21) }

→ randomly | using formula

run 5 epochs/iterations

Δw } change in weights and
 Δb } bias
 -or-
 $\uparrow$ { derivative given w
 derivative given b

Figure 6.16 Backpropagation algorithm

Q dataset containing one record

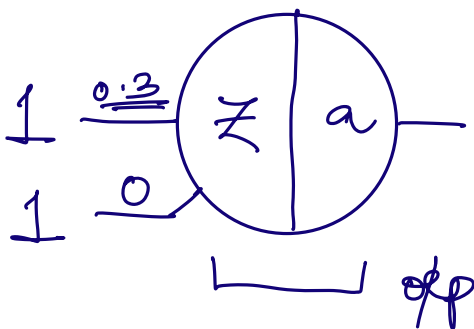
x	y
1	1

$$w_x = 0.3$$

$$b = 0$$

$a = \text{sigmoid}$

error = as per backprop algo



η or learning rate = 0.9
 (1)

Solution:

Epoch: 1

Step 1: forward propagation

$$\begin{aligned} z &= w_1 x_1 + b \\ &= (0.3)(1) + 0 \\ &= \underline{\underline{0.3}} \end{aligned}$$

$$a = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-0.3}} = 0.5744 = o_j$$

Step 2: Error calc.

$$\begin{aligned} \text{Err}_j &= o_j (1 - o_j) (T_j - o_j) \\ &= 0.5744 (1 - 0.5744) (1 - 0.5744) \\ &= 0.1040 \end{aligned}$$

actual label
(ground truth)

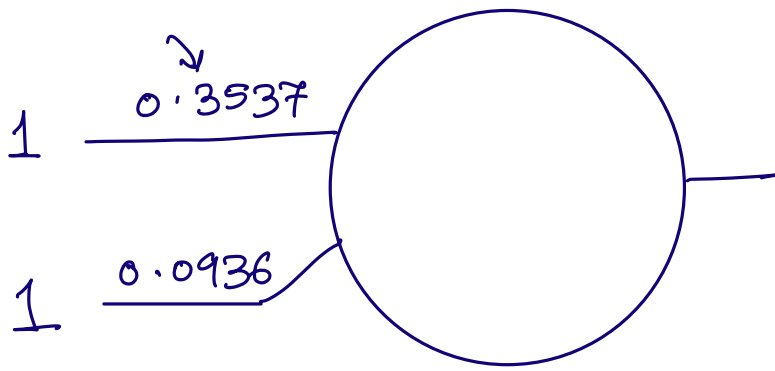
Step 3: Calc gradients and update w & b
(Δw & Δb)

$$\begin{aligned} \Delta w &= \eta \cdot \text{Err}_j \cdot o_i \\ &= (0.9) (0.1040) (0.5744) \\ &= 0.0537 \end{aligned}$$

$$\begin{aligned} w &= w + \Delta w \\ &= 0.3 + 0.0537 \\ &= 0.3537 \end{aligned}$$

$$\begin{aligned} \Delta b &= \eta \cdot \text{Err}_j \\ &= (0.9) (0.1040) \\ &= 0.0936 \end{aligned}$$

$$\begin{aligned} b &= b + \Delta b \\ &= 0 + 0.0936 \\ &= 0.0936 \end{aligned}$$



Epoch 2:

step 1 : forward prop

$$\begin{aligned}
 z &= w_1 x_1 + b \\
 &= (0.3537)(1) + (0.0936)(1) \\
 &= 0.4473
 \end{aligned}$$

$$a = \frac{1}{1 + e^{-0.4473}} = 0.6099 = O_j$$

step 2: Error calc.

$$\begin{aligned}
 Err_j &= O_j (1 - O_j) (T_j - O_j) \\
 &= 0.6099 (1 - 0.6099) (1 - 0.6099) \\
 &= 0.0927 //
 \end{aligned}$$

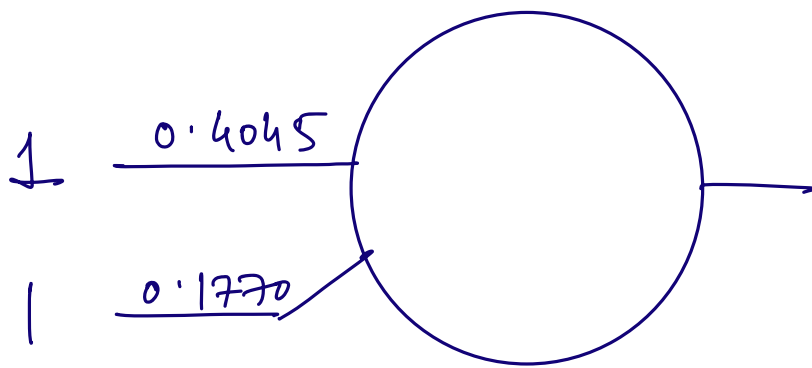
step 3: calc gradient
 Δw & Δb

$$\begin{aligned}\Delta w &= \eta \cdot \text{Err}_j \cdot O_i \\ &= (0.9)(0.0927)(0.6099) \\ &= 0.0508\end{aligned}$$

$$\begin{aligned}\Delta b &= \eta \cdot \text{Err}_j \\ &= (0.9)(0.0927) \\ &= 0.0834\end{aligned}$$

$$\begin{aligned}w &= w + \Delta w \\ &= 0.3537 + 0.0508 \\ &= 0.4045\end{aligned}$$

$$\begin{aligned}b &= b + \Delta b \\ &= 0.0936 + 0.0834 \\ &= 0.1770\end{aligned}$$



You will do epoch 3 to epoch 10
 [Stop if error not reducing]